Homework 1

n26140765 林郁家

1. Environment

Linux x86-64, GNU C++ 15.2.0, CMake 4.2.3, OpenGL 3.3 core profile, GLFW 3.4.0, and system GLM 1.0.1. The supplied code also uses GLAD, stb_image 2.25, tinyobjloader 0.9.20, and Dear ImGui 1.77.

2. Method Description

OpenGLShader.cpp

create() uses glCreateShader(type_) to create a vertex, fragment or geometry shader. compileFromSource() passes the GLSL source to glShaderSource and compiles it with glCompileShader. compileStatus() queries GL_COMPILE_STATUS through glGetShaderiv and returns whether compilation succeeded. tidy() calls glDeleteShader and clears the object ID.

OpenGLTexture.cpp

create() generates a texture ID with glGenTextures. bind() and release() bind the texture or ID 0 to GL_TEXTURE_2D. bindBuffer() sets minification, magnification and S/T wrapping with glTexParameteri, uploads pixels with glTexImage2D, and generates mipmaps with glGenerateMipmap. Pixel unpack alignment is temporarily set to 1 for RGB rows of arbitrary width. setMagnificationFilter(), setMinificationFilter() and setWrapOption() update their corresponding texture parameters. tidy() deletes the texture with glDeleteTextures.

OpenGLBufferObject.cpp

create() allocates a buffer ID with glGenBuffers. bind() binds that ID to type_ using glBindBuffer, where type_ selects a vertex buffer or index buffer. allocateBufferData() calls glBufferData(type_, size, data, usagePattern_) to allocate GPU storage and upload the supplied data to the bound buffer. release() unbinds the target by binding ID 0. tidy() releases GPU storage with glDeleteBuffers and clears the ID.

OpenGLShaderProgram.cpp

create() creates a program with glCreateProgram. attachShader() attaches a compiled shader with glAttachShader. link() calls glLinkProgram, and linkStatus() queries GL_LINK_STATUS with glGetProgramiv. use() activates the linked program with glUseProgram. enableAttributeArray() and disableAttributeArray() control vertex attribute arrays. mapAttributePointer() uses glVertexAttribPointer to define each attribute's size, type, normalization, stride and byte offset. destroyShaders() detaches and destroys owned shaders. destroyProgram() deletes the program with glDeleteProgram.

OpenGLVertexArrayObject.cpp

create() generates a vertex array object with glGenVertexArrays. bind() selects it with glBindVertexArray so attribute settings and the element-buffer binding are stored in that VAO. release() binds VAO 0. tidy() calls glDeleteVertexArrays and clears the object ID.

3. Bonus

The sample provides camera position, look-at target, background color and wireframe controls. Added controls include a 20-90 degree field-of-view slider, automatic camera orbit and reset view.

Model selection switches between the supplied Utah teapot and an original UV-mapped cube with 24 face-local vertices and 12 triangles. Texture selection switches between the supplied UV image and an original 256 x 256 blue/gold checkerboard. Either texture can be used with either model.

4. How to run the program

```
cd hw1/sample_code
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
cd build/bin/Release
./Homework01 \
  "./resources/model/Utah_teapot_(solid)_texture.obj" \
  "./resources/texture/uv.png" \
  "./Shader/BasicVertexShader.vs.glsl" \
  "./Shader/BasicFragmentShader.fs.glsl"
```

5. AI Usage

Model: GPT-6.1 Sol Medium

Token usage: 3,724,291 total tokens (input: 3,703,121; output: 21,170; cached input included in input: 3,563,648). Recorded at 2026-09-30 16:34:40 UTC+08:00.